

ML Specification Report — playground-series-s3e3

Employee Attrition — Binary Classification • AIDE agent (20-step tree search, Claude Sonnet via shim)

All scores are taken from the run's `facts_aide.json`. Tree structure, per-step plans and per-node execution times come from `logs/2-amphibian-spectral-hummingbird/journal.json`; hyperparameters from `best_solution.py` and the run's `config.yaml`; metric/target metadata from the competition `config.yaml`. Dataset shapes are counted directly from the run's own `input/` directory.

Contents	
Employee Attrition — Binary Classification • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Optimization specification	6
Inference procedures	6
Evaluation & Benchmarking	7

Overview

The task is to predict whether an employee leaves the company (`Attrition`) from a synthetic HR table derived from the IBM attrition dataset, scored by **ROC AUC (maximize)**. AIDE searched it as a tree of executable scripts over **20 steps — 17 scored, 3 buggy** — lifting a LightGBM draft at **0.8206** to a champion **local OOF AUC of 0.8463** found at **step 17**, three steps before the budget ended. The champion is a five-model ensemble (HistGradientBoosting, RandomForest, LogisticRegression, XGBoost, CatBoost) evaluated under repeated 5×3 stratified CV, isotonically calibrated, converted to percentile ranks, and combined by 20,000-sample Dirichlet search plus coordinate-ascent refinement.

This is the weakest episode of the batch (s3e1, s3e3, s3e5, s3e7, s3e9) on the leaderboard: **public 0.88484, private 0.86863, rank 491/666 — the 26.4th percentile**, behind both other lanes (my agent 0.87303, NVIDIA 0.89537), and the score table records `local_winner = nvidia` and `lb_winner = nvidia`. The diagnosis is visible inside the run itself and is developed below: on 1,677 training rows AIDE built a weight-fitting machine with far more degrees of freedom than the data supports, and then — knowingly — selected the variant whose calibration step leaks.

Why it matters. Attrition prediction drives real retention budgets, and it is the archetype of the small, noisy, mixed-type HR table where the gap between a defensible validation protocol and a leaderboard-chasing one is widest. For the agent comparison this episode is the clearest case of an agent optimizing its own validation signal rather than the task.

Data

Purpose of Data. Predict a binary attrition label from 30-odd employee attributes — demographics, compensation, tenure, satisfaction ordinals and job categoricals — with `Attrition` as target, `id` as key, and ROC AUC as the metric. **Data Format** is **mixed-type tabular CSV**: mostly integer-coded ordinals and counts, plus seven string categorical columns. **Data Volume**, counted from the run's `input/` directory, is **1,677 training rows × 35 columns** and **1,119 test rows × 34 columns**. This is a very small dataset — smaller than a single fold of most Playground episodes — and essentially every finding below follows from that fact. The `input/` directory contained only `train.csv`, `test.csv` and `sample_submission.csv`, so this run is a clean from-raw-data search.

Data Quality was never profiled (**no EDA artifact exists** — AIDE ran no dedicated analysis step), but two defects were found by AIDE while writing code and are handled explicitly in the champion. First, three columns are constant and carry no signal: `EmployeeCount`, `StandardHours`, `Over18` are dropped. Second, `Education` and `JobLevel` contain corrupted out-of-range values — AIDE's own report notes examples such as 15 and 7 in columns whose valid range is 1–5 — and are clipped with `clip(upper=5)`. Missing-value counts, duplicate counts and train↔test drift were never measured by this run.

Annotation Guidelines. The label is binary. Because the metric is AUC, submissions are **scores, not decisions**: only the ordering of the 1,119 test rows matters, and no probability threshold is ever applied.

Feature Set. The champion's label-encoded matrix is 38 columns wide:

$$35 \text{ (raw columns)} - 3 \text{ (constant columns dropped)} - 2 \text{ (id, Attrition)} = 30 \text{ retained raw features}$$
$$30 + 8 \text{ (engineered ratios/composites)} = 38 \text{ features}$$

Group	Features
Retained raw (30)	demographics and compensation (<code>Age</code> , <code>DailyRate</code> , <code>MonthlyIncome</code> , <code>MonthlyRate</code> , <code>HourlyRate</code> , <code>PercentSalaryHike</code> , <code>DistanceFromHome</code> , <code>StockOptionLevel</code>), tenure counters (<code>TotalWorkingYears</code> , <code>YearsAtCompany</code> , <code>YearsInCurrentRole</code> , <code>YearsSinceLastPromotion</code> , <code>YearsWithCurrManager</code> , <code>NumCompaniesWorked</code> , <code>TrainingTimesLastYear</code>), satisfaction/rating ordinals (<code>JobSatisfaction</code> , <code>EnvironmentSatisfaction</code> , <code>RelationshipSatisfaction</code> , <code>WorkLifeBalance</code> , <code>JobInvolvement</code> , <code>PerformanceRating</code> , <code>Education</code> , <code>JobLevel</code>), and seven categoricals (<code>BusinessTravel</code> , <code>Department</code> , <code>EducationField</code> , <code>Gender</code> , <code>JobRole</code> , <code>MaritalStatus</code> , <code>OverTime</code>)

Group	Features
Engineered ratios (8)	IncomeToAge , TenureRatio , RoleTenureRatio , PromotionGap , ManagerTenureRatio , SatisfactionComposite , IncomePerJobLevel , CompaniesPerYear

Encoding is model-specific and both variants are built up front: a **label encoding fitted on `concat(train, test)`** for HistGradientBoosting, RandomForest, LogisticRegression and CatBoost (the last using them as native `cat_features`), and a **one-hot expansion** of the same seven columns for XGBoost, aligned across splits with `align(join="left", fill_value=0)`. Fitting the label encoder on train+test is transductive but harmless here — it only guarantees that unseen test categories get a consistent code. Out-of-fold smoothed target encoding was tried at step 14 and rejected (0.84062 against the 0.84104 incumbent).

Splitting strategy. The champion uses `RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)` — 15 folds in total, stratified because the positive class is a minority. OOF predictions are accumulated per row and divided by the number of times the row was held out, so the OOF vector is the 3-repeat average. Switching from a single 5-fold split to repeated CV was, on AIDE's own accounting, the largest structural gain in the run (step 1 → step 3, 0.82394 → 0.83573). The leaderboard was exercised: the champion's submission was scored publicly and privately.

Models & Architecture

Purpose of Architecture. Maximize AUC — i.e. produce a good ordering of employees by attrition risk on a 1,677-row training set. **Architecture Type** is a **five-member heterogeneous ensemble with a two-stage combination layer**: per-fold isotonic calibration, then a percentile-rank transform, then a weighted average whose weights come from Dirichlet random search refined by coordinate ascent. The five members are deliberately diverse in inductive bias — two histogram GBDTs (HistGradientBoosting, XGBoost), one ordered-boosting GBDT with native categorical splits (CatBoost), one bagged tree ensemble (RandomForest) and one linear model (LogisticRegression on standardized features).

Input Format is two parallel dense matrices — the label-encoded one for four members, the one-hot one for XGBoost. **Input Dimension** is **38 features** in the label-encoded matrix; the one-hot matrix is wider by the expansion of the seven categorical columns, and its exact width is **not recorded**.

Architecture Description. HistGradientBoosting: `max_iter 300, learning_rate 0.05, max_depth 4, l2_regularization 1.0`, native categoricals, internal early stopping on a 15 % validation fraction with patience 20. RandomForest: 400 trees, `max_depth 8, min_samples_leaf 3`. LogisticRegression: `C 0.5, max_iter 1000`, on `StandardScaler`-transformed features. XGBoost: 500 estimators, `learning_rate 0.03, max_depth 4, subsample 0.8, colsample_bytree 0.8, reg_lambda 1.0`, early stopping 30 on fold AUC. CatBoost: 600 iterations, `learning_rate 0.03, depth 6, l2_leaf_reg 3.0`, native `cat_features`, early stopping 40. Each member's fold seed is the fold index.

The combination layer is where the run's character lies. Each member's validation and test probabilities are passed through an `IsotonicRegression` **fitted on that member's in-sample training predictions**

(see the Training section — this is the leak AIDE later diagnosed and kept). The calibrated vectors are then jointly percentile-ranked over [validation rows || full test set] per fold and averaged across the 15 folds. Weights are searched over the 5-simplex with three hand-seeded candidates plus **20,000 Dirichlet(0.8) samples**, scored by OOF AUC in both probability and rank space, then refined by **coordinate ascent** over six shrinking step sizes (0.08 down to 0.0025), and finally the better of the two spaces is selected. Step 17's analysis records rank space winning, with weight mass concentrated on HistGradientBoosting and CatBoost.

Model Complexity is best counted in fits:

5 members × 5 folds × 3 repeats = 75 model fits

plus a weight search that evaluates ROC AUC on the full OOF vector 20,003 times per space and again on every coordinate-ascent probe.

Training procedures

Search trajectory

Step	Parent	What AIDE changed	OOF AUC
0	—	draft: LightGBM, label-encoded, 5-fold	0.8206
1	—	draft: XGBoost, one-hot + engineered ratios	0.82394
2	1	+ LightGBM, fixed 50/50 average	0.82308
3	1	repeated 5×3 stratified CV	0.83573
4	3	CatBoost with native categoricals	0.81883
5	—	draft: PyTorch MLP with entity embeddings	buggy — ModuleNotFoundError
6	—	draft: PyTorch MLP (second attempt)	buggy — ModuleNotFoundError
7	—	draft: PyTorch MLP (third attempt, + outlier clipping)	buggy — ModuleNotFoundError
8	7	repair → HGB + RF + LR fixed-weight blend, repeated CV	0.83929
9	8	+ XGBoost as 4th member, grid-searched weights	0.83944
10	9	logistic stacking meta-learner with pairwise terms	0.83327
11	9	+ CatBoost as 5th member, Dirichlet weight search	0.84104
12	6	repair → single-split HGB	0.81598
13	5	repair → single-split HGB	0.80836
14	11	+ out-of-fold smoothed target encoding	0.84062
15	11	+ per-fold isotonic calibration	0.84355
16	15	+ rank-space blending	0.84617

Step	Parent	What AIDE changed	OOF AUC
17	16	+ coordinate-ascent weight refinement (champion)	0.8463
18	17	honest calibration via 80/20 holdout	0.83455
19	17	honest calibration via inner 3-fold CV	0.84111

The tree opened with the configured **5 root drafts** (steps 0, 1, 5, 6, 7) and reached **depth 7**; the champion sits at depth 6 on the lineage $7 \rightarrow 8 \rightarrow 9 \rightarrow 11 \rightarrow 15 \rightarrow 16 \rightarrow 17$. **9 of the 17 scored steps set a new incumbent.** Total improvement:

$$0.8463 - 0.8206 = 0.0257 \quad (3.1319 \% \text{ relative})$$

Three steps were lost to the same environmental failure. Steps 5, 6 and 7 are three independent attempts to draft a PyTorch MLP with learned categorical embeddings, and all three died instantly on `ModuleNotFoundError: No module named 'torch'` — 3 of 20 steps, 15 % of the budget, spent rediscovering that the execution image has no PyTorch. AIDE's draft policy never learned across drafts; only the repair nodes (8, 12, 13) adapted, by replacing the network with scikit-learn gradient boosting. In fairness, this misfire was productive: the repair at step 8 introduced the HistGradientBoosting + RandomForest + LogisticRegression trio that became the champion's backbone, so the tree's best branch descends from a failed neural draft.

The honest problem with this run is the last four steps. The champion's gain over the five-model Dirichlet blend is

$$0.8463 - 0.84104 = 0.00526$$

and every point of it comes from the combination layer — not from data, features or base models:

$$\begin{aligned} 0.84355 - 0.84104 &= 0.00251 && (\text{per-fold isotonic calibration, step 15}) \\ 0.84617 - 0.84355 &= 0.00262 && (\text{rank-space blending, step 16}) \\ 0.8463 - 0.84617 &= 0.00013 && (\text{coordinate-ascent refinement, step 17}) \end{aligned}$$

At step 18 AIDE correctly diagnosed that its calibration is leaky: the isotonic map is fitted on each model's in-sample training predictions, which are overconfident, and then applied to the validation rows that produce the OOF score being optimized. It wrote two leak-free replacements. Both scored worse:

$$\begin{aligned} 0.8463 - 0.83455 &= 0.01175 && (80/20 \text{ holdout calibration}) \\ 0.8463 - 0.84111 &= 0.00519 && (\text{inner 3-fold OOF calibration}) \end{aligned}$$

and AIDE's report concludes that the leaky variant "empirically produced better-ranked blends", keeping it as champion. That is the wrong inference from the right experiment: on 1,677 rows, a 5-weight simplex searched 20,000 times plus coordinate ascent, scored against an OOF vector contaminated by in-sample calibration, is a selection procedure fitting its own validation signal. The leaderboard agrees — 491/666, the 26.4th percentile, the worst placement in this batch.

One piece of context for reading "step 0": this journal is the second leg of the run, started with `initial_journal` pointing at a 5-node journal from an earlier leg, so AIDE's memory at step 0 already contained five prior attempts on this competition.

Optimization specification

The **Loss Function** is binary cross-entropy for the learned members — CatBoost `Logloss`, XGBoost `binary:logistic`, HistGradientBoosting's and LogisticRegression's log loss — while **model selection is on ROC AUC**, including the blend-weight search, which optimizes the non-differentiable AUC objective directly by random sampling and coordinate ascent. The **Optimization Algorithm** is gradient boosting for three members, bagging for the RandomForest and L-BFGS for the LogisticRegression; on top sit two derivative-free optimizers (Dirichlet sampling, coordinate ascent). The **Learning Rate** is the boosting shrinkage: 0.05 for HistGradientBoosting, 0.03 for XGBoost and CatBoost (N/A for RandomForest and LogisticRegression). There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by early stopping, patience 20/30/40 per member, not a schedule) and **no Batch Size** (N/A — all five learners are full-dataset fitters).

Training Duration for the whole search was **458.4 s of executed wall-clock across 20 nodes** (mean 22.9 s, max 164.5 s), and no node hit the 1800 s harness timeout. The three torch drafts each cost about a millisecond, so effectively the entire budget bought 17 real experiments. **Training Memory Consumption Limits** are **not recorded**.

Transfer Learning is N/A (no pretrained weights; the competition forbids them); its analogue is **journal memory**, and the trace is explicit in the plans — step 11 reasons that "CatBoost alone underperformed but was never tried as an ensemble member (only in isolation)", which is a memory-driven mutation, and it produced the run's largest single ensemble gain. **Data Augmentation** is N/A (tabular); the analogues are the eight engineered ratio features and **repeat bagging** over the three CV repeats.

Reproducibility Standards are moderate and unverified: `np.random.seed(0)` at module scope, `RepeatedStratifiedKFold(random_state=42)`, per-member seeds set to the fold index, Dirichlet search seeded at 123, coordinate ascent seeded at 7 (probability space) and 17 (rank space). But **AIDE runs no replay or rebuild gate** — 0.8463 is what one execution printed, and nothing re-ran the champion to confirm it.

Inference procedures

Decision Threshold is N/A — the metric is AUC, so ordering is submitted and no threshold is ever applied. The **post-processing chain is unusually long for a tabular submission** and is part of the model rather than an afterthought: per-fold isotonic calibration of each member's test probabilities, a percentile-rank transform computed jointly over that fold's validation rows and the full test set, averaging of those ranks over 15 folds, and finally the weighted rank-space combination. Two consequences deserve stating. First, the submitted values are **average percentile ranks in [0, 1]**, not probabilities — legitimate under AUC but meaningless as risk estimates. Second, the joint val+test

ranking makes each test row's score depend on the rest of the test set: the pipeline is transductive. The only remaining step is an id-order realignment against `sample_submission.csv`.

Inference Duration is **not recorded** separately — test scoring happens inside each fold's loop, so it is included in the champion node's runtime. **Inference Memory Consumption Limits** are **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Score all 1,119 test employees by attrition risk and submit a two-column CSV in `sample_submission.csv` order. **Performance Metrics.** ROC AUC, maximized. AIDE's local trajectory runs $0.8206 \rightarrow 0.83573 \rightarrow 0.83944 \rightarrow 0.84104 \rightarrow 0.8463$ across the draft, repeated CV, the four-model blend, the five-model Dirichlet blend and the calibrated rank-space champion. Local and leaderboard disagree in an informative direction:

$$\begin{aligned} 0.86863 \text{ (private)} - 0.8463 \text{ (local OOF)} &= 0.02233 \\ 0.88484 \text{ (public)} - 0.86863 \text{ (private)} &= 0.01621 \end{aligned}$$

The local OOF number is pessimistic relative to the leaderboard, which is what one expects when the training set is small and the models are refit on progressively larger folds — so the failure here is not an inflated local estimate but a **badly ranked** one: AIDE's local ordering of candidates rewarded the leaky-calibration variant. The submission placed **491/666, the 26.4th percentile**.

Performance Benchmarking. All three lanes submitted to the same private leaderboard:

Lane	Approach	Private AUC	Note
NVIDIA	reproduce-agent, public-kernel replication	0.89537	<code>lb_winner</code>
My agent	from-scratch GBDT pool + tree search	0.87303	ahead of AIDE by 0.0044
AIDE	20-step tree search, calibrated rank-space 5-model blend	0.86863	last

$$\begin{aligned} 0.89537 - 0.86863 &= 0.02674 \\ 0.87303 - 0.86863 &= 0.0044 \end{aligned}$$

AIDE finishes third, and the gap to NVIDIA (0.02674) is slightly larger than everything the twenty-step search achieved locally (0.0257) — the whole run would have had to happen twice over to close it. The mechanism is not compute — this run used 458.4 s and never timed out — and not model choice; all three lanes are GBDT ensembles. It is validation discipline. AIDE spent its last five productive steps refining a combination layer on 1,677 rows, and when its own leak audit at step 18 said the champion's calibration was invalid, it read the resulting score drop as evidence for the leak rather than evidence that the OOF objective had stopped measuring generalization. `facts_aide.json` records both `local_winner` and `lb_winner` as `nvidia`; on this episode the two agree.